

DEPARTMENT OF COMPUTER APPLICATIONS
FACULTY OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

A PROJECT SYNOPSIS

ON

**SMARTSHOP: A DIGITAL SHOP
MANAGEMENT, POINT-OF-SALE (POS) &
CUSTOMER KHATA LEDGER SYSTEM**

Submitted in partial fulfillment of the requirements for the degree of
MASTER OF COMPUTER APPLICATIONS (MCA)
Semester – III (Mini Project)

Submitted By:

Ajit Kumar

Roll No: 2504280140008

Enrollment No: EN2504280140008

MCA 3rd Semester

Under the Guidance of:

Project Guide / Faculty Coordinator

Assistant Professor

Department of Computer Applications

Academic Session: 2026–2027

CERTIFICATE

This is to certify that the Project Synopsis entitled "**SmartShop: A Digital Shop Management, Point-of-Sale (POS) & Customer Khata Ledger System**" submitted by **Ajit Kumar** (Roll No: 2504280140008, Enrollment No: EN2504280140008) in partial fulfillment of the requirements for the award of the degree of **Master of Computer Applications (MCA – 3rd Semester)** from the Department of Computer Applications, is an authentic record of project work carried out under my supervision and guidance.

To the best of my knowledge, the matter embodied in this project synopsis has not been submitted to any other University or Institute for the award of any other degree or diploma.

Project Guide / Supervisor

Department of Computer Applications

Head of Department (HOD)

Department of Computer Applications

DECLARATION

I hereby declare that the project synopsis entitled "**SmartShop: A Digital Shop Management, Point-of-Sale (POS) & Customer Khata Ledger System**" is an original work done by me under the guidance of our Project Guide in the Department of Computer Applications.

I further declare that this synopsis represents my genuine understanding and design of the proposed MCA 3rd semester mini project, and it has not formed the basis for the award of any degree, diploma, fellowship or other similar title to any candidate of any University.

Date: _____

Place: _____

Ajit Kumar

Roll No: 2504280140008

MCA 3rd Semester

ACKNOWLEDGEMENT

I express my deepest gratitude and sincere thanks to our respected Project Guide, Department of Computer Applications, for providing continuous guidance, valuable suggestions, and constant encouragement throughout the preparation of this project synopsis.

I also extend my sincere thanks to our Head of Department (HOD) and all faculty members of the Department of Computer Applications for providing the necessary laboratory facilities, academic support, and constructive feedback.

Finally, I would like to thank my parents, team members, and fellow classmates whose continuous moral support, discussions, and help made the completion of this synopsis possible.

Ajit Kumar
MCA 3rd Semester

TABLE OF CONTENTS

S. No.	Topic / Section Title	Page No.
—	Certificate	i
—	Declaration	ii
—	Acknowledgement	iii
—	List of Figures	v
—	List of Tables	vi
—	Abstract / Project Summary	vii
1.	Introduction	1
2.	Problem Statement	2
3.	Existing System	3
4.	Limitations of Existing System	3
5.	Proposed System	4
6.	Objectives of the Project	5
7.	Scope of the Project (Current vs Future Scope)	5
8.	Users and Roles	6
9.	Modules of the System	7
10.	Functional Requirements (FR-01 to FR-12)	8
11.	Non-Functional Requirements	9
12.	Technology Stack	10
13.	Development Methodology (Agile)	10
14.	System Architecture	11
15.	Use Case Diagram	12
16.	DFD Level 0 (Context Level Diagram)	13
17.	DFD Level 1 (Functional Decomposition)	14
18.	ER Diagram	15
19.	Database Design (MongoDB Collections)	16
20.	Hardware and Software Requirements	17
21.	Project Development Timeline	18
22.	Expected Outcome	18
23.	Limitations of Current System	19
24.	Future Scope (OTP Login, Password Reset, WhatsApp)	19
25.	Conclusion	20

26.	References / Bibliography	21
-----	---------------------------	----

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1	SmartShop 3-Tier System Architecture Diagram	11
Figure 2	UML Use Case Diagram (Admin & Customer Actions)	12
Figure 3	Data Flow Diagram — Level 0 (Context Level Diagram)	13
Figure 4	Data Flow Diagram — Level 1 (Functional Decomposition)	14
Figure 5	Entity-Relationship (ER) Diagram	15

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Technology Stack Specifications	10
Table 2	Hardware & Software Requirements (SRS)	17
Table 3	Project Development Timeline Schedule	18

ABSTRACT / PROJECT SUMMARY

In the local neighborhood retail ecosystem, small grocery and provision shopkeepers heavily depend on manual paper notebooks (Khata bahi) to record daily customer credit sales ('Udhar'), cash billing, and inventory stock. Concurrently, customers face the practical inconvenience of not knowing whether a desired grocery item is in stock without physically walking to the shop. Furthermore, handwritten records frequently suffer from mathematical calculation errors, torn pages, lost credit dues, and complete lack of payment proof transparency.

SmartShop is an integrated fullstack web application built using the **MERN Stack (MongoDB, Express.js, React 19, Node.js)** to solve these real-world retail challenges. The system enables customers to check real-time product catalog availability, prices, and stock indicators from their homes on mobile devices before visiting the store. At the store counter, the shopkeeper utilizes an intuitive Point-of-Sale (POS) billing interface to process sales via Full Cash, Full UPI, Full Credit (Khata), or Partial Payment splits. The digital ledger tracks borrowed amounts, repayment due dates, and borrow limits. Customers can view itemized receipts and submit payment claims (Cash/UPI screenshot proofs/Razorpay) which the admin verifies. Product returns automatically restock inventory and deduct customer debt. Built with ACID transaction rollbacks, SmartShop delivers an efficient, transparent, and modern solution for retail shop operations.

Keywords: *Shop Management System, Point of Sale (POS), Digital Khata Ledger, MERN Stack, React 19, MongoDB Transactions, Customer Self-Service Portal.*

1. INTRODUCTION

1.1 Background of the Project

Neighborhood retail stores, grocery shops, and local provision merchants represent the primary source of daily household commodities across Indian towns and cities. One of the most distinctive features of this retail sector is the informal credit mechanism ('Udhar' / 'Khata'), where regular customers take household goods daily and settle their accumulated debt weekly or monthly. While this credit facility fosters strong customer loyalty, managing these financial transactions on physical paper notebooks is highly cumbersome, error-prone, and inefficient.

With the rapid penetration of smartphones and digital payment infrastructure (such as UPI), retail customers increasingly expect transparency and digital accessibility. However, most existing commercial Point-of-Sale (POS) software solutions are designed exclusively for large supermarket chains—they focus solely on instant card/cash payments, require expensive proprietary hardware terminals, and completely lack built-in customer credit khata management.

1.2 Domain of the Project

SmartShop belongs to the domain of **Enterprise Web Application Development & Point of Sale (POS) Information Systems**. It bridges customer-facing self-service information with administrative shopkeeper workflows using contemporary web technologies.

1.3 Purpose and Need of the System

The fundamental purpose of SmartShop is to replace chaotic, manual paper-based retail management with a centralized, reliable, and accessible cloud web system. Specifically, the system addresses three core operational needs:

- 1. Customer Product Availability Check:** Allows customers to browse store inventory and check whether items are in stock from their home before physically visiting the store.
- 2. Fast POS Counter Billing:** Enables shopkeepers to generate multi-item bills in seconds supporting cash, UPI, credit, and partial payment combinations.
- 3. Automated Khata Ledger & Payment Tracking:** Maintains an accurate ledger of customer borrowings, due dates, borrow limits, and verifies payment proofs to avoid financial disputes.

2. PROBLEM STATEMENT

In traditional local retail stores, two critical problems severely degrade daily operations:

Problem 1 (Customer Inconvenience): Customers frequently make trips to the neighborhood shop only to discover that necessary products are out of stock. There is currently no simple mechanism for a customer to verify product availability and pricing from home without making phone calls or physically walking to the shop.

Problem 2 (Ledger Errors & Disputes): Shopkeepers record credit sales in handwritten notebooks. These paper ledgers are prone to arithmetic errors during manual daily addition, physical page tearing, and ink fading. Moreover, customers have zero visibility into their itemized dues, leading to frequent arguments during monthly settlements.

3. EXISTING SYSTEM

In the existing retail setup, the workflow is entirely manual and unorganized:

- **Stock Verification:** Shopkeeper manually checks physical shelves when a customer asks for an item.
- **Billing:** Calculated on a handheld calculator and written on paper slips.
- **Credit Recording:** Amount is scribbled under the customer's page in a paper bahi-khata.
- **Payments:** Customer gives cash or transfers via UPI without any permanent itemized link to specific credit entries.

4. LIMITATIONS OF EXISTING SYSTEM

- **Heavy Manual Work:** Time-consuming record keeping and daily ledger total recalculations.
- **Risk of Data Loss:** Physical register damage from moisture, fire, tearing, or misplacement.
- **Zero Customer Self-Service:** Customers cannot check their debt history or stock availability online.
- **No Stock Synchronization:** Stock levels are not decremented when items are sold on credit.
- **Disputed Returns:** Product returns require manual recalculations across multiple register pages.

5. PROPOSED SYSTEM

SmartShop is proposed as a centralized, fullstack digital shop management and point-of-sale platform designed to overcome the limitations of manual registers:

- **Home Stock Availability Check:** Customers can log into their mobile portal from home, search categories (Grocery, Snacks, Dairy, Personal Care), check prices, and verify whether items are in stock before going to the shop.
- **Multi-Mode Counter Billing:** Shopkeeper can quickly bill items and choose Full Cash, Full UPI, Full Credit (Khata), or Partial Payment splits.
- **Automated Digital Khata:** Credit balances are calculated automatically with repayment due dates and customer borrow limits.
- **Payment Claims Verification:** Customers can submit cash receipts or upload UPI screenshot proofs with UTR IDs, which the shopkeeper verifies and approves with one click.
- **Full & Partial Returns:** Returned items are automatically restocked into inventory and deducted from customer debt in an atomic transaction.

6. OBJECTIVES OF THE PROJECT

1. To develop a web-based Point of Sale (POS) system for fast counter billing supporting cash, UPI, credit, and partial payments.
2. To provide customers with a real-time product catalog to check store item availability and pricing from home.
3. To digitize customer credit accounts ('Khata') with automated due dates, borrow limits, and repayment tracking.
4. To enable payment verification where customers submit cash/UPI proofs and shopkeepers review and approve them.
5. To implement automated product return workflows that restore inventory stock and reduce debt balances atomically.
6. To ensure high database reliability by implementing multi-document ACID transaction rollbacks on write failures.

7. SCOPE OF THE PROJECT

7.1 Current Scope (Implemented in Current Project):

- Multi-role authentication (Admin, SuperAdmin, Customer) with JWT and single-use password reset tokens.

- Complete inventory catalog management with Cloudinary image hosting.
- POS sales counter with cash/UPI/credit/partial billing and atomic stock deduction.
- Digital khata credit ledger with borrow limits, due dates, and FIFO payment settlements.
- Payment claims approval dashboard and Razorpay payment gateway integration.
- Full and partial product return engine with stock restoration.

7.2 Future Scope (Planned in Subsequent Versions):

- OTP-based mobile number login for customer portal.
- OTP-based secure password reset verification.
- Automated WhatsApp and SMS invoice receipts and payment due date reminders.
- Thermal POS receipt printer integration for instant physical billing slips.

8. USERS AND ROLES

The system defines three distinct user roles with specific permissions:

1. **Store Admin / Shopkeeper:** Manages daily operations including counter POS sales, inventory stock updates, customer credit limits, payment approvals, and return processing.
2. **Retail Customer:** Accesses self-service portal to browse product catalog availability from home, view itemized purchase invoices, track pending khata debt, and submit payment claims.
3. **SuperAdmin:** Platform administrator with system-wide oversight, admin account creation, and global store configuration control.

9. MODULES OF THE SYSTEM

- **9.1 Authentication & Profile Module:** Manages user registration, login via email/phone/username, role segregation, JWT token generation, and single-use password reset invalidation.
- **9.2 Product Catalog & Inventory Module:** Provides product CRUD operations, category filters (Grocery, Dairy, Snacks, etc.), stock quantity tracking, price management, and Cloudinary CDN image uploads.
- **9.3 POS Sales & Billing Engine:** Enables shopkeeper to build counter bills, select customer, deduct stock atomically, and settle via Cash, UPI, Credit (Khata), or Partial Payment splits.
- **9.4 Digital Khata / Credit Ledger Module:** Maintains customer debt records, tracks borrowed vs paid amounts, evaluates maximum borrow limits, and monitors repayment due dates.
- **9.5 Payment Reconciliation Module:** Allows customers to submit cash claims or upload UPI payment screenshots with UTR IDs. Shopkeeper verifies and approves claims, which decrements debt atomically.
- **9.6 Returns & Balance Adjustment Module:** Handles full or partial item returns against historical invoices, automatically restocks inventory, and offsets outstanding credit debt.
- **9.7 Activity & Audit Logging Module:** Maintains a chronological audit log of all store transactions and admin actions for complete transparency.

10. FUNCTIONAL REQUIREMENTS

The functional requirements of SmartShop are specified in standard format:

FR-01: User Registration

The system shall allow new customers to create an account using their name, email, phone number, and password.

FR-02: User Authentication

The system shall authenticate registered users using email, phone, or username with secure password verification and JWT token issuance.

FR-03: Product Catalog & Stock View

The system shall allow customers to browse categories, view product pricing, and check real-time stock availability from home.

FR-04: Inventory Management

The system shall allow authorized store admins to add, update, view, and soft-delete products with Cloudinary image uploads.

FR-05: POS Sales Billing

The system shall allow store admins to create sales invoices with catalog or custom items and choose Cash, UPI, Credit, or Partial payment.

FR-06: Atomic Stock Deduction

The system shall atomically decrement product stock upon sale completion and mark products out-of-stock when inventory reaches zero.

FR-07: Borrow Limit Enforcement

The system shall check customer outstanding credit against their maximum borrow limit before approving new credit sales.

FR-08: Customer Khata Ledger

The system shall maintain an individual ledger for each customer displaying total borrowed amount, amount paid, and net pending dues.

FR-09: Payment Claims Submission

The system shall allow customers to submit Cash payment claims or upload UPI payment screenshots with UTR transaction IDs.

FR-10: Admin Payment Verification

The system shall provide admins with a verification queue to inspect payment proofs and approve or reject claims with atomic balance updates.

FR-11: Product Returns Processing

The system shall allow admins to process full or partial item returns, automatically restocking inventory and reducing customer credit dues.

FR-12: Single-Use Password Reset

The system shall generate secure, time-bound password reset tokens and invalidate them immediately after password update.

11. NON-FUNCTIONAL REQUIREMENTS

- **Security:** Passwords encrypted with bcrypt hashing, stateless JWT session tokens, and security response headers (XSS, frameguard, nosniff).
- **Reliability & Atomicity:** Multi-document write operations wrapped in MongoDB ACID transactions to guarantee zero corrupted financial ledgers.
- **Performance:** Lightweight Vite build and indexed database queries ensuring API response times under 200ms.
- **Usability:** Modern responsive UI built with TailwindCSS and glassmorphism styling, fully usable on smartphones, tablets, and desktop computers.
- **Availability:** 24/7 cloud accessibility for customer ledger inquiry and store operations.

12. TECHNOLOGY STACK

Table 1 summarizes the technologies selected for the development of SmartShop:

Layer / Tier	Technology / Tool	Reason for Selection
Frontend (Customer)	React 19, Vite, TailwindCSS	Component-based architecture, ultra-fast rendering, and responsive UI.
Frontend (Admin)	React 19, Lucide Icons, Vite	Clean administrative POS interface, charts, and verification views.
Backend Server	Node.js, Express.js (ESM)	Asynchronous non-blocking I/O, RESTful API architecture, modular routes.
Database	MongoDB, Mongoose ORM	Flexible JSON document store supporting multi-document ACID transactions.
Authentication	JWT, Bcrypt.js	Stateless cryptographic token auth with salted password hashing.
Media Storage	Cloudinary CDN & Multer	Cloud image optimization and secure hosting for product catalog and proofs.
Payment Gateway	Razorpay API	Seamless UPI, Netbanking, and Card payment clearance for credit settlements.
Version Control & IDE	Git, GitHub, VS Code	Industry-standard version tracking, branch management, and development IDE.

Table 1: Technology Stack Specifications

13. DEVELOPMENT METHODOLOGY

SmartShop was developed following the **Agile Software Development Methodology**. Agile was selected because it enables iterative feature building, frequent testing, and rapid incorporation of feedback across sprints:

- Sprint 1 (Requirement Analysis & Database Schema):** Gathered local store pain points and designed 7 relational MongoDB collections.
- Sprint 2 (Backend REST APIs & Auth):** Developed JWT authentication, RBAC middleware, and product/sales controllers.
- Sprint 3 (Frontend POS & Customer Portal):** Built responsive React 19 interfaces for counter billing, catalog view, and khata ledger.
- Sprint 4 (Payments, Returns & Transactions):** Integrated Razorpay, payment verification queue, and atomic MongoDB transactions.
- Sprint 5 (Testing & Security Hardening):** Verified 0-error production builds, rate limiting, and single-use password reset tokens.

14. SYSTEM ARCHITECTURE

SmartShop implements a classic **3-Tier Architecture** consisting of the Presentation Tier, Application Tier, and Database/Cloud Services Tier, as illustrated in Figure 1:

SmartShop — 3-Tier System Architecture

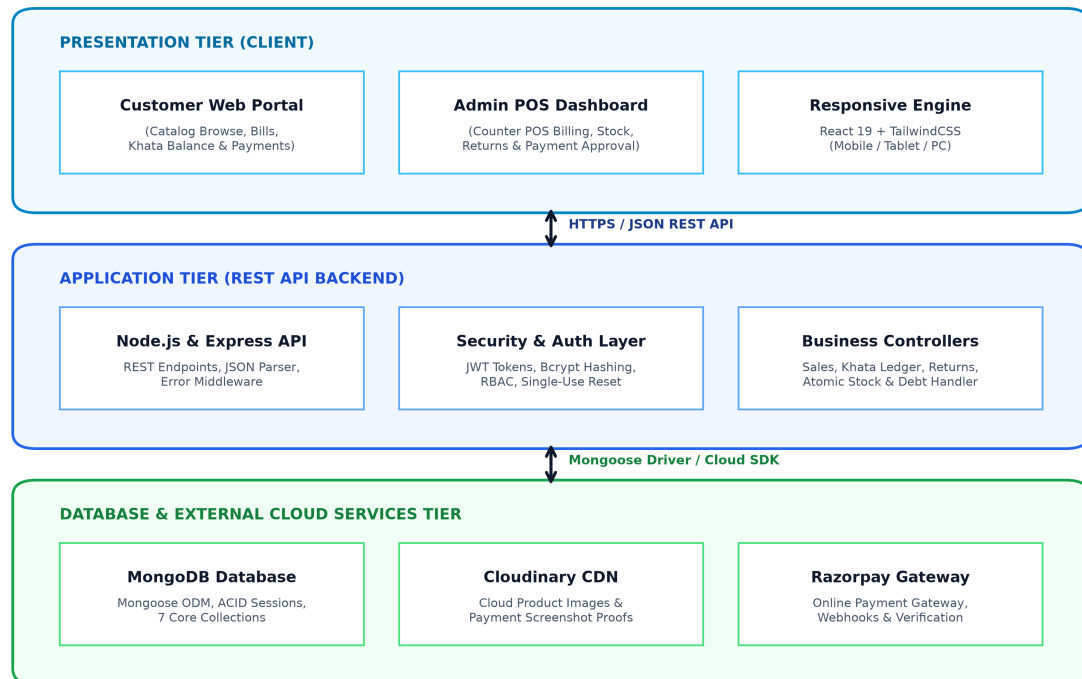


Figure 1: SmartShop 3-Tier System Architecture Diagram

- **Tier 1 (Presentation):** Single Page Applications (SPAs) built in React 19 for Customer and Admin portals communicating via JSON HTTP requests.
- **Tier 2 (Application):** Express.js server executing business controllers, role-based authorization, and MongoDB transactional sessions.
- **Tier 3 (Database & Cloud):** MongoDB database storing persistent documents, alongside Cloudinary CDN and Razorpay API.

15. USE CASE DIAGRAM

Figure 2 depicts the UML Use Case Diagram showing the primary actors (Store Admin and Retail Customer) and their interactions with the major functional use cases of the SmartShop system:

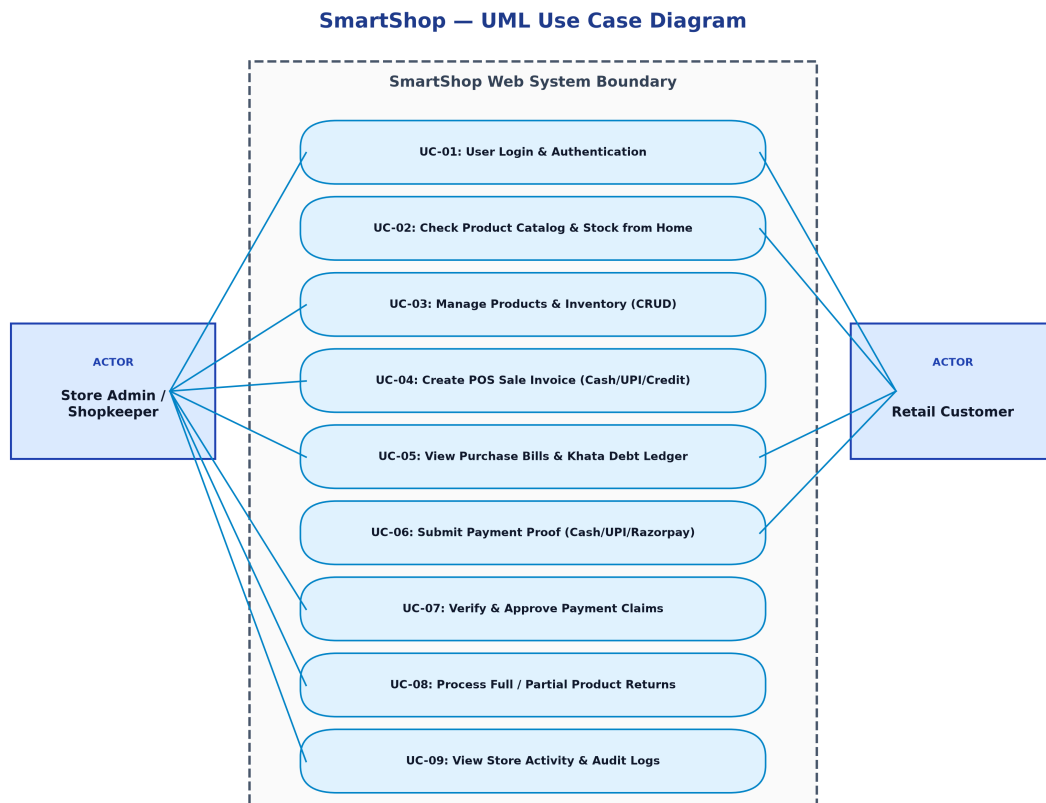


Figure 2: UML Use Case Diagram for SmartShop

The diagram illustrates that both actors share authentication and ledger inspection use cases, while operational POS billing, inventory CRUD, returns, and payment verification are restricted to the Store Admin actor.

16. DATA FLOW DIAGRAM — LEVEL 0 (CONTEXT DIAGRAM)

The DFD Level 0 Context Diagram (Figure 3) defines the high-level boundary of the SmartShop system, showing all external entities and their major data inflows and outflows:

SmartShop — DFD Level 0 (Context Level Diagram)

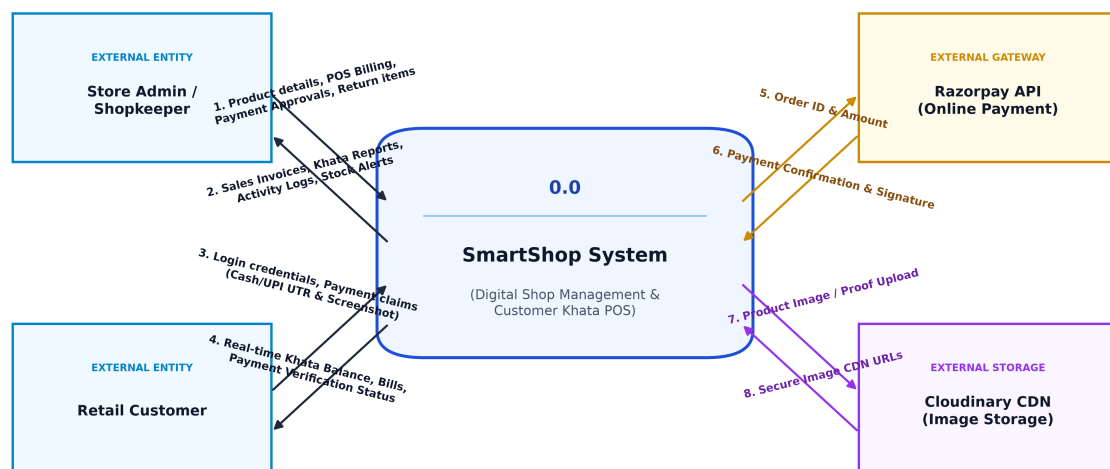


Figure 3: Data Flow Diagram — Level 0 (Context Level Diagram)

- **Store Admin:** Sends product catalog details, counter sales items, payment approvals, and return actions; receives sales receipts, khata reports, and activity logs.
- **Retail Customer:** Sends login credentials, cash/UPI payment claims with screenshot proofs; receives real-time product stock view, purchase invoices, and balance clearance status.
- **Razorpay API & Cloudinary:** Facilitates online payment signature verification and secure cloud image asset hosting.

17. DATA FLOW DIAGRAM — LEVEL 1 (DECOMPOSITION)

Figure 4 decomposes the main system into **6 core functional sub-processes** and maps their read/write interactions with the **7 MongoDB data stores (D1 to D7)**:

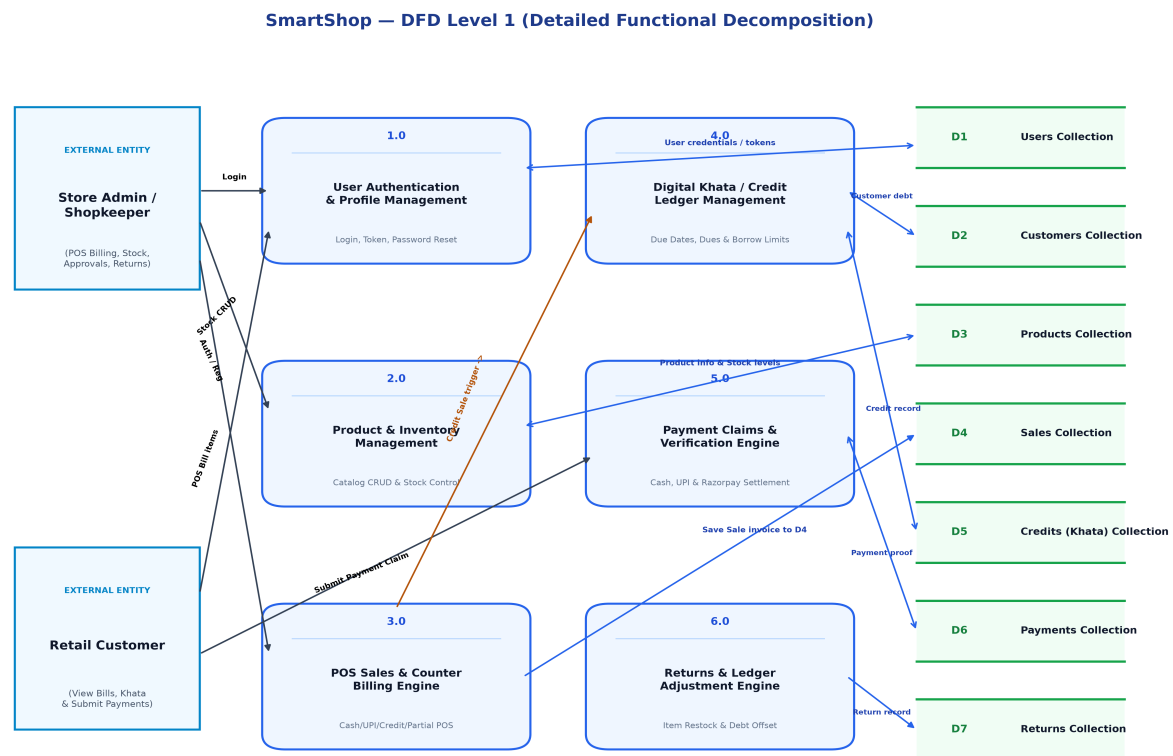


Figure 4: Data Flow Diagram — Level 1 (Functional Decomposition)

- **1.0 Auth & Profile:** Interacts with D1 (Users) and D2 (Customers).
- **2.0 Products & Stock:** Manages product catalog and stock levels in D3 (Products).
- **3.0 POS Sales Engine:** Reads D3, writes invoices to D4 (Sales), and triggers credit in 4.0.
- **4.0 Digital Khata Ledger:** Manages customer debt balances in D2 (Customers) and D5 (Credits).
- **5.0 Payment Verification:** Writes claims to D6 (Payments) and decrements dues in D5 and D2.
- **6.0 Returns Engine:** Restores stock in D3, records returns in D7, and offsets debt in D5.

18. ENTITY-RELATIONSHIP (ER) DIAGRAM

Figure 5 illustrates the primary entities, attributes, primary keys, foreign key references, and cardinality relationships across the SmartShop database structure:

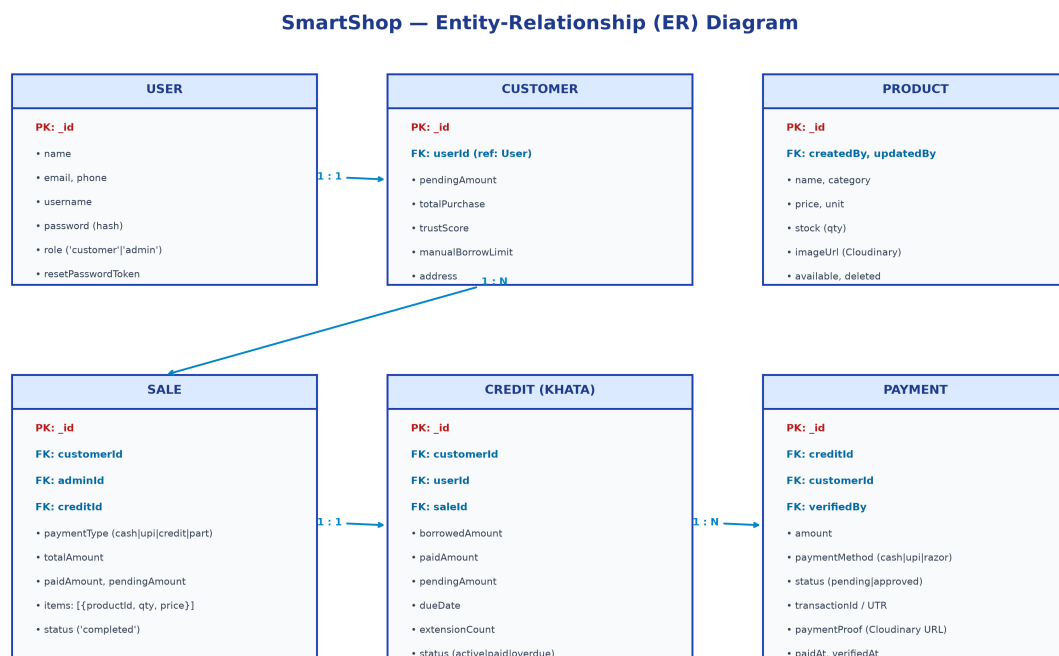


Figure 5: Entity-Relationship (ER) Diagram

- **User to Customer:** 1-to-1 relationship linking login credentials to customer financial records.
- **Customer to Sale:** 1-to-Many relationship where one customer can make multiple purchases.
- **Sale to Credit:** 1-to-1 optional relationship linking credit-based sales to individual credit loans.
- **Credit to Payment:** 1-to-Many relationship where a single credit debt can be settled via multiple partial payments.

19. DATABASE DESIGN (COLLECTION SCHEMAS)

The system implements 7 primary MongoDB collections structured with Mongoose schemas:

Users Collection (D1)

Field	Type	Constraints & Description
_id	ObjectId	Primary Key (Auto generated)
name	String	Full user name (Required)
email / phone	String	Unique login identifiers (Sparse)
password	String	Bcrypt hashed password (Required)
role	String	Enum: 'customer' 'admin' 'superadmin'
resetPasswordToken	String	Single-use token with expiration timestamp

Customers Collection (D2)

Field	Type	Constraints & Description
_id	ObjectId	Primary Key
userId	ObjectId	Foreign Key -> Users._id (Required)
pendingAmount	Number	Total outstanding credit balance
totalPurchase	Number	Cumulative lifetime purchase amount
trustScore	Number	Computed customer repayment rating
manualBorrowLimit	Number	Max credit debt ceiling configured by admin

Products Collection (D3)

Field	Type	Constraints & Description
_id	ObjectId	Primary Key
name	String	Product item name (Required)
category	String	Category: Grocery, Snacks, Dairy, etc.
price	Number	Unit retail price (Required)
stock	Number	Current available inventory quantity
imageUrl	String	Cloudinary CDN hosted asset URL
available	Boolean	Auto set to false when stock = 0
deleted	Boolean	Soft-delete flag to protect sales history

Sales Collection (D4)

Field	Type	Constraints & Description
_id	ObjectId	Primary Key
customerId	ObjectId	Foreign Key -> Customers._id
items	Array	[{ productId, quantity, price, total }]
totalAmount	Number	Invoice bill total amount
paymentType	String	Enum: 'cash' 'upi' 'credit' 'partial'
paidAmount	Number	Amount paid instantly at counter
pendingAmount	Number	Amount added to customer credit ledger

Credits Collection (D5)

Field	Type	Constraints & Description
_id	ObjectId	Primary Key
customerId	ObjectId	Foreign Key -> Customers._id
borrowedAmount	Number	Original credit loan amount
paidAmount	Number	Cumulative amount settled towards credit
pendingAmount	Number	Net remaining debt balance
dueDate	Date	Repayment deadline
status	String	Enum: 'active' 'partially_paid' 'paid' 'overdue'

Payments Collection (D6)

Field	Type	Constraints & Description
_id	ObjectId	Primary Key
creditId	ObjectId	Foreign Key -> Credits._id
amount	Number	Payment submission amount
paymentMethod	String	Enum: 'cash' 'upi' 'razorpay'
status	String	Enum: 'pending' 'approved' 'rejected'
transactionId	String	UPI UTR ID or Razorpay Payment ID
paymentProof	String	Cloudinary image URL of receipt screenshot

Returns Collection (D7)

Field	Type	Constraints & Description
_id	ObjectId	Primary Key
saleId	ObjectId	Foreign Key -> Sales._id
returnedItems	Array	[{ productId, quantity, refundAmount }]
refundAmount	Number	Total calculated return refund value
balanceDeducted	Number	Amount offset from customer credit debt
processedBy	ObjectId	Foreign Key -> Users._id (Admin)

20. HARDWARE AND SOFTWARE REQUIREMENTS

Table 2 outlines the minimum and recommended system requirements:

Hardware Requirements	Software Requirements
• Processor: Intel Core i3 / AMD Ryzen 3 or higher • RAM: 4 GB (8 GB Recommended) • Storage: 500 MB free hard disk space • Display: Responsive on Mobile, Tablet & Desktop • Network: Active 4G / Broadband Internet	• Operating System: Windows 10/11, Linux, macOS • Runtime: Node.js (v18.x / v20.x) • Database: MongoDB Atlas / Local v6.0+ • Web Browser: Chrome, Edge, Firefox, Safari • Version Control: Git & GitHub

Table 2: Hardware & Software Requirements (SRS)

21. PROJECT DEVELOPMENT TIMELINE

Table 3 outlines the 12-week development schedule for the project:

Week	Activity / Milestone	Outcome / Deliverable
Week 1	Requirement Analysis & Literature Survey	Requirement Specification Document
Week 2	System Requirements Specification (SRS)	Finalized SRS Document
Week 3	System Architecture & Diagram Design	Use Case, DFD & Architecture Diagrams
Week 4	Database Schema Design & ER Modeling	MongoDB Mongoose Collections
Week 5–6	Backend API Development & Auth	REST API Endpoints & JWT Security
Week 7–8	Frontend Customer & Admin UI	React 19 Responsive Web Pages
Week 9	System Integration (Razorpay, Cloudinary)	Working Fullstack Integration
Week 10	Testing & Transaction Rollback Hardening	Zero-error Builds & Validation
Week 11	Deployment & GitHub Repository Setup	Live Cloud Deployment
Week 12	Documentation & Synopsis Preparation	Final Synopsis & Presentation PPT

Table 3: Project Development Timeline Schedule

22. EXPECTED OUTCOME

Upon successful deployment, SmartShop delivers direct benefits to both shopkeepers and customers:

- **For Retail Customers:** Eliminates unnecessary physical store trips by allowing them to check real-time stock and prices from home. Provides 24/7 transparent access to itemized purchase invoices and khata dues.
- **For Store Shopkeepers:** Reduces counter billing time by over 80%, eliminates paper register damage, prevents forgotten credit dues, and enforces credit limits automatically.
- **Academic Outcome:** Demonstrates practical mastery of fullstack MERN development, database transactions, and responsive interface engineering.

23. LIMITATIONS OF CURRENT SYSTEM

The current prototype has the following realistic limitations:

- **Internet Dependency:** Requires an active internet connection for live cloud synchronization.
- **Manual Camera Barcode:** Product addition is currently catalog-based and does not yet scan barcodes via hardware laser scanners.
- **Single Store Focus:** Designed for single-store retail operations rather than multi-branch enterprise chains in the initial version.

24. FUTURE SCOPE

The following features are planned for subsequent development phases:

1. **OTP-Based Customer Login:** Implementation of passwordless phone number OTP login via SMS gateway for rapid mobile access.
2. **OTP-Based Password Reset:** Enhancing password recovery with time-sensitive SMS/Email OTP verification.
3. **Automated WhatsApp / SMS Invoices:** Automatic delivery of bill receipts and payment due date reminders directly to customers' WhatsApp numbers.
4. **Thermal POS Receipt Printing:** Adding direct USB and Bluetooth thermal printer drivers for physical paper invoice slips.

25. CONCLUSION

SmartShop provides a practical, robust, and modern digital platform that solves persistent operational bottlenecks in traditional retail shop management. By enabling customers to check stock availability from home, streamlining counter POS sales with multi-payment mode flexibility, digitizing customer credit khata records, and automating return adjustments, the project successfully replaces error-prone paper notebooks with an ACID-compliant cloud web system. The project strictly satisfies all academic requirements for an MCA 3rd Semester Mini Project and establishes a solid foundation for future retail software enhancements.

26. REFERENCES / BIBLIOGRAPHY

- [1] V. Subramanian, *Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node*, 2nd ed., Apress, 2019.
- [2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.
- [3] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed., McGraw-Hill Education, 2014.
- [4] React 19 Official Documentation, "React – A JavaScript library for building user interfaces," <https://react.dev/>
- [5] MongoDB Inc., "MongoDB Manual: Multi-Document ACID Transactions," <https://www.mongodb.com/docs/manual/core/transactions/>
- [6] Express.js Foundation, "Fast, unopinionated, minimalist web framework for Node.js," <https://expressjs.com/>
- [7] Tailwind Labs, "Tailwind CSS – Rapidly build modern websites without ever leaving your HTML," <https://tailwindcss.com/>
- [8] Razorpay Software Private Limited, "Razorpay Payment Gateway API Documentation," <https://razorpay.com/docs/>